Relatório de Entrega: Gerenciamento e Monitoramento de Logs

Implementação prática de Observabilidade, Tratamento Estruturado de Falhas e Métricas Operacionais

Projeto: EstoqueRevisao (Fábrica de Software)	**Branch Dedicada:** atividade/gerenciamento-monitoramento-logs
Endpoint Escolhido: Catálogo de Produtos (`/api/v1/products`)	**Status: 100% Implementado, Testado e Documentado**

1. Objetivo e Escopo da Atividade

O objetivo da atividade consistiu em desenvolver e integrar ao projeto de software da Fábrica um subsistema completo de **observabilidade e monitoramento de logs**. A solução permite registrar eventos em tempo real, filtrar e consultar ocorrências por múltiplos atributos, analisar métricas de erro na aplicação e acionar alertas automáticos em cenários críticos, mantendo isolamento estrito na branch designada para a atividade.

2. Tratamento de Falhas (Esperadas e Inesperadas)

Foi selecionado o controlador de **Produtos (`/api/v1/products`)**. Todas as falhas retornam payloads padronizados com códigos HTTP semânticos via `GlobalExceptionFilter`, garantindo conformidade com a segurança de dados (zero vazamento de *stack traces*, senhas ou detalhes de infraestrutura).

Tipo de Falha	Cenário Simulado	Código HTTP / Erro	Nível Log
Esperada (Client Handled)	Código de produto duplicado no cadastro	**409 Conflict** `PRODUCT_CODE_ALREADY_EXISTS`	**WARN**
Esperada (Client Handled)	Busca por ID de produto inexistente	**404 Not Found** `PRODUCT_NOT_FOUND`	**WARN**
Inesperada (Infraestrutura)	Indisponibilidade / queda do PostgreSQL	**503 Service Unavailable** `DATABASE_UNAVAILABLE`	**ERROR**
Inesperada (Aplicação)	Exceção interna não tratada (crash)	**500 Internal Server Error** `INTERNAL_ERROR`	**FATAL**

3. Registro Estruturado de Logs em JSON e Sanitização LGPD

Os eventos são emitidos em formato **JSON canônico estruturado** contendo os atributos essenciais de rastreabilidade, passando obrigatoriamente pela rotina de higienização `redactSensitive()` que substitui recursivamente tokens, senhas e chaves criptográficas por `[REDACTED]`.

Resiliência com Buffer Circular de Fallback: A persistência é realizada primariamente na tabela relacional `system_logs` do PostgreSQL. Se o banco de dados cair ou ficar inacessível, o `SystemLogsRepository` intercepta a falha e grava o registro em um buffer de memória de até 2000 eventos, impedindo travamentos e garantindo que o próprio log de falha de conexão continue consultável.

4. Gerenciamento, Consulta e Interface Web SPA

A solução disponibiliza uma suíte RESTful completa para consulta integrada com uma interface visual reativa em React:

Método e Rota	Propósito e Filtros Suportados
GET `/api/v1/logs`	Consulta paginada com busca livre (`search`), nível (`level`), código (`errorCode`), contexto (`context`), `eventId`, `startDate` e `endDate`.
GET `/api/v1/logs/:id`	Recuperação dos detalhes completos de uma ocorrência única via UUID.
GET `/api/v1/logs/stats/count-by-error`	Agrupamento quantitativo de ocorrências por tipo de erro para identificação rápida.
GET `/api/v1/logs/metrics`	Consolidação de métricas operacionais, taxa percentual de erros e status dos alertas.

Painel Web Integrado (`LogsMonitoringPage.tsx`): Acessível no menu lateral do sistema, exibe banner com badge de alerta em tempo real, cards de contagem, botões de disparo de simulação com 1 clique (inclusive teste de rajada de erros), ranking de erros recorrentes, formulário de filtros dinâmicos e modal com visualização formatada do JSON.

5. Monitoramento, Métricas e Regras de Alerta

REGRA DE ALERTA OPERACIONAL IMPLEMENTADA:
- **Condição de Incidente Crítico:** Mais de **10 erros críticos (`ERROR` ou `FATAL`) em uma janela de 5 minutos**.
- **Status CRITICAL:** Flag `alert.triggered = true`, emitindo mensagem de incidente operacional imediato.
- **Status WARNING:** Entre 1 e 9 erros críticos nos últimos 5 minutos (alerta preventivo).
- **Status NORMAL:** Zero falhas críticas no período de observação.

Métricas Acompanhadas: Contagem por nível de criticidade (`INFO`, `WARN`, `ERROR`, `FATAL`); Taxa global de falhas (`errorRatePercentage`); e Lista ranqueada de erros recorrentes (`topRecurringErrors`) com percentual de impacto, quantidade absoluta e carimbo do último evento.

6. Validação Automatizada e Evidências

Todas as funcionalidades foram validadas por suítes de testes unitários e de integração automatizados (**46 test suites aprovados, 329 testes unitários**):

Suíte de Teste	Cenários Verificados	Resultado
`products-failure-simulation.spec.ts`	Simulação 409 (conflito), 404 (inexistente), 503 (banco) e 500 (crash) com payloads padronizados.	**100% PASS**
`system-logs.service.spec.ts`	Geração de JSON estruturado, mascaramento de senhas/tokens, cálculo de métricas e disparo de alerta (>10 erros/5min).	**100% PASS**
`system-logs.controller.spec.ts`	Endpoints de consulta paginada, filtros combinados, estatísticas por código e recuperação por UUID.	**100% PASS**
`executar-demonstracao.cjs`	Script completo de demonstração executando todas as chamadas HTTP e gerando relatório de evidências.	**100% SUCESSO**

7. Arquivos Entregues no Repositório

- **Backend:** `apps/backend/src/modules/system-logs/` (Entity, Repository, Service, Controller, Module, DTOs);
- **Migrations:** `1791590400000-system-logs-monitoring.ts` (tabela e índices otimizados);
- **Frontend:** `apps/frontend/src/LogsMonitoringPage.tsx` integrado com rotas e navegação;
- **Documentação e Evidências:** Pasta `entregas/atividade-gerenciamento-monitoramento-logs/` contendo `DOCUMENTACAO.md`, `RESPOSTA.md`, `exemplos-requisicoes-respostas.http` e `resultado-demonstracao.txt`.